Name: KEY ID:

Q1	Q2	Q3	Q4	Q5	Q6	SUM
/25	/15	/15	/15	/30	/20	/100

+5 pts BONUS

CSE 2046 Analysis of Algorithms

Spring 2020

Online Homework 2

Solution and Submission Notes

- Textbooks, slides, and notes are open.
- Internet usage is not allowed; however, computers and smartphones may be used for viewing course materials provided for this course.
- Write your name, surname and student ID on the top of each exam solution page.
- Show all your work.
- Write the following sentence on the top of the first page with your handwriting and sign it: "On my honor, I have neither given nor received any unauthorized assistance on this examination."
- Either print this PDF file (6 pages) or use blank A4 pages for your solutions. Solve all the questions with your handwriting.
- Scan all the solution pages to a single PDF file named "firstname_lastname.pdf". Do not include raw photos, use an Android/IOS scanner application. All the writings should be very readable. Page layouts should be portrait, not landscape.
- Upload the PDF file to https://ues.marmara.edu.tr system before the deadline. Give at least 15-20 minutes for scanning and upload process.

Name: KEY ID:

On my honor, I have neither given nor received any unauthorized assistance on this examination.

Name Surname:

Signature:

Q-1. (25 pts) (a – 7 pts) How many **character comparisons** will the **Horspool algorithm** make in searching the pattern **110110** in the binary text of 1000 zeros? **(Do not forget to show the shift table)**

Shift table:

1→1

0→3

2pts

0 0 0 0 0 0 0 0 0 0

1 1 0 1 1 0 #1

1 1 0 1 1 0 #1

2 comparison 3 shift. 2pts

$(1000-4)\cdot\frac{2}{3}=664$ Comparisons.

3pts

$6+3k\leq 1000$

max value: 999

(b – 9 pts) How many **character comparisons** will the **Boyer-Moore** algorithm make in searching the pattern **110110** in the binary text of 1000 zeros? **(Show bad-symbol and good-suffix tables)**

k	good suffix	d_2
1	1 1 0 1 1 **0**	6
2	1 1 0 1 **1 0**	6
3	1 1 0 **1 1 0**	3
4	1 **1 0 1 1 0**	3
5	**1 1 0 1 1 0**	3

4pts

0 0 0 0 0 0 0 0 0 0 0 0 0 0

1 1 0 1 1 0 #1

2 Comparison 6 shift.

2pts

Bad symbol:

1→1 0→3

$d_1=3-1=2$

$d_2=6$

$6+6k\leq 1000$

max value: 996

$\frac{996}{3}=332$ Comparisons.

3pts

(c – 9 pts) For a binary text of *n* zeros, is it possible to have a pattern where bad symbol shift is more than good suffix shift, i.e. $d_1>d_2$? If yes, show an example. If no, prove it.

We talk about good suffix shift only when there is a suffix match, i.e last character of the pattern is zero.

If there is more than one zero in the suffix, then bad symbol shift would be 1, so it cannot be greater than d_2.

If there is just one zero at the end of pattern, then

x x x x x 1 0 good suffix shift would be at least the distance of last character to the rightmost zero (other than the last char). Bad symbol shift cannot be greater than this value (actually it is this value minus one).

Name: KEY ID:

15

Q-2. (~~20~~ pts) Consider a staircase with n steps. Steps are indexed from 0 to n-1. Each step i has some non-negative cost, cost[i]. A robot wants to climb these stairs. Once it pays the cost, it can either climb one or two steps. Describe a dynamic programming algorithm to find minimum cost to reach the top of the floor. Robot can either start from the step with index 0, or the step with index 1. Following are some example inputs and outputs.

Example 1:

Input: cost = [10, 15, 20]

Output: 15

Explanation: Minimum cost is obtained when the robot starts on cost[1], pay that cost and go to the top.

Example 2:

Input: cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]

Output: 6

Explanation: Minimum cost is obtained when the robot starts on cost[0], and only step on 1s, skipping cost[3].

Hint: Define f[i] as the final cost to climb to the top from step i. Find a recurrence relation for f[i].

$f[i] = cost[i] + \min(f[i+1], f[i+2])$

$f[n-1] = cost[n-1]$

$f[n-2] = cost[n-2]$.

Recurrence relation

```
for (i = n-3; i >= 0; i--)
    f[i] = cost[i] + min(f[i+1], f[i+2])

return (min(f[0], f[1]));
```

Alternative:

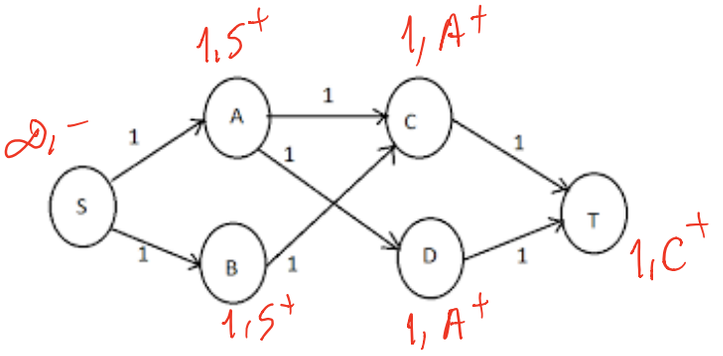
here f[i] is defined as cost of climbing from bottom to i.

```
f[0] = cost[0]
f[1] = cost[1]
for (i = 2, i <= n-1, i++)
    f[i] = min(f[i-1] + cost[i], f[i-2] + cost[i]);
return (min(f[n-1], f[n-2]));
```

Name: KEY _____ **ID:** _____

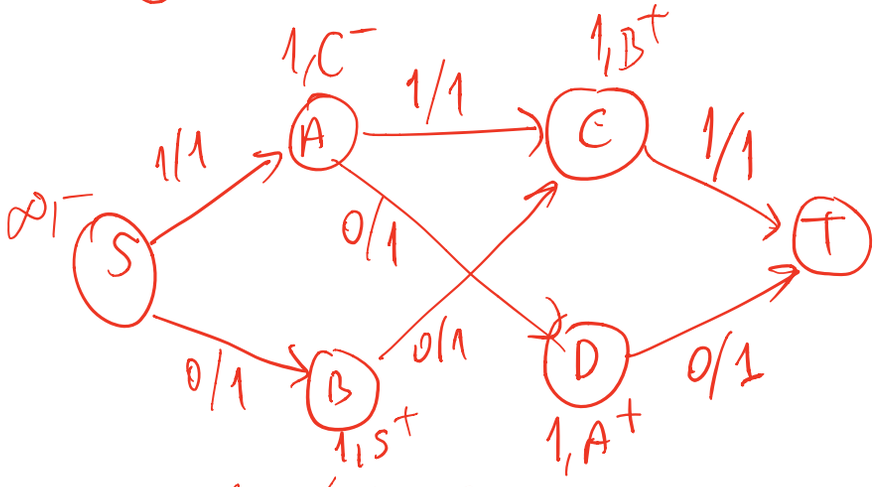
Q-3. (15 pts) Apply the **shortest augmenting path algorithm** to find the **maximum flow** (source is 1 and sink is 6). *While applying BFS,* ***visit the neighbors in alphabetical order.*** *Show the* ***labels*** *and* ***BFS queue*** *clearly in each iteration.*

Also give the **minimum cut** that is found by your algorithm.



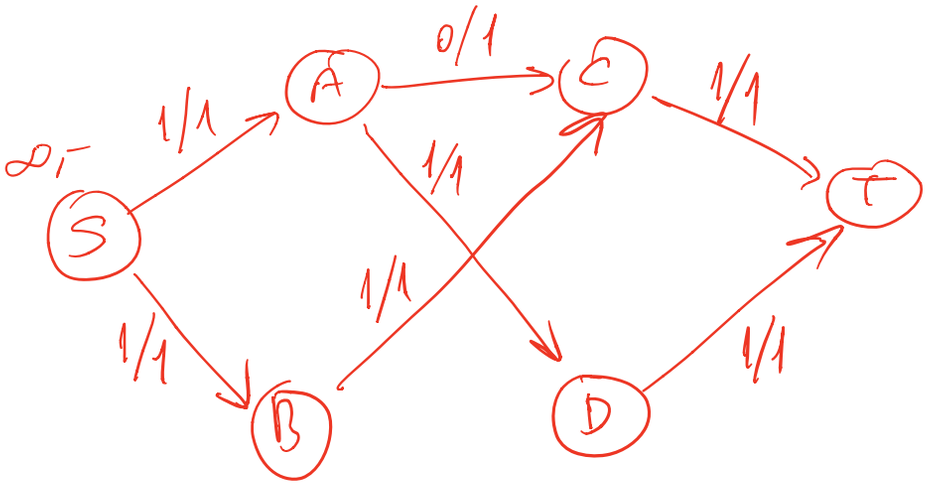
Q: ~~S~~ ~~A~~ ~~B~~ C D

S → A → C → T



Q: ~~S~~ ~~B~~ ~~C~~ ~~A~~ D T

S → B → C ← A → D → T



Q: S

minimum cut found by the algorithm:

{(S,A),(S,B)}

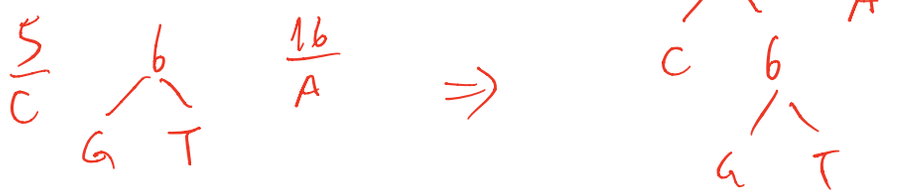
Name: KEY ID:

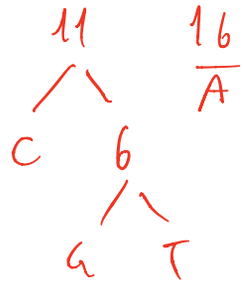
Q-4. (15 pts) Use Huffman Coding to encode the following DNS string and find the compression ratio.

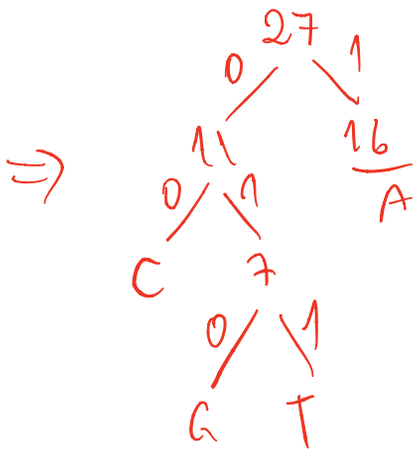
AGATAACCATCGAAACAAAATCAAATA

(Note: You are expected to extract the statistical knowledge from the input itself.)

16A, 4T, 2G, 5C







A: 1

T: 011

G: 010

C: 00

Total codeword length: $1.16 + 3.2 + 3.4 + 2.5$

$= 16 + 6 + 12 + 10 = 44$

Fixed length encoding: $(16+4+2+5) \times 2 = 54$

Compression rate: $\frac{54-44}{54} * 100 = \frac{1000}{54}\%$

$= 18,52\%$

Name: KEY ID:

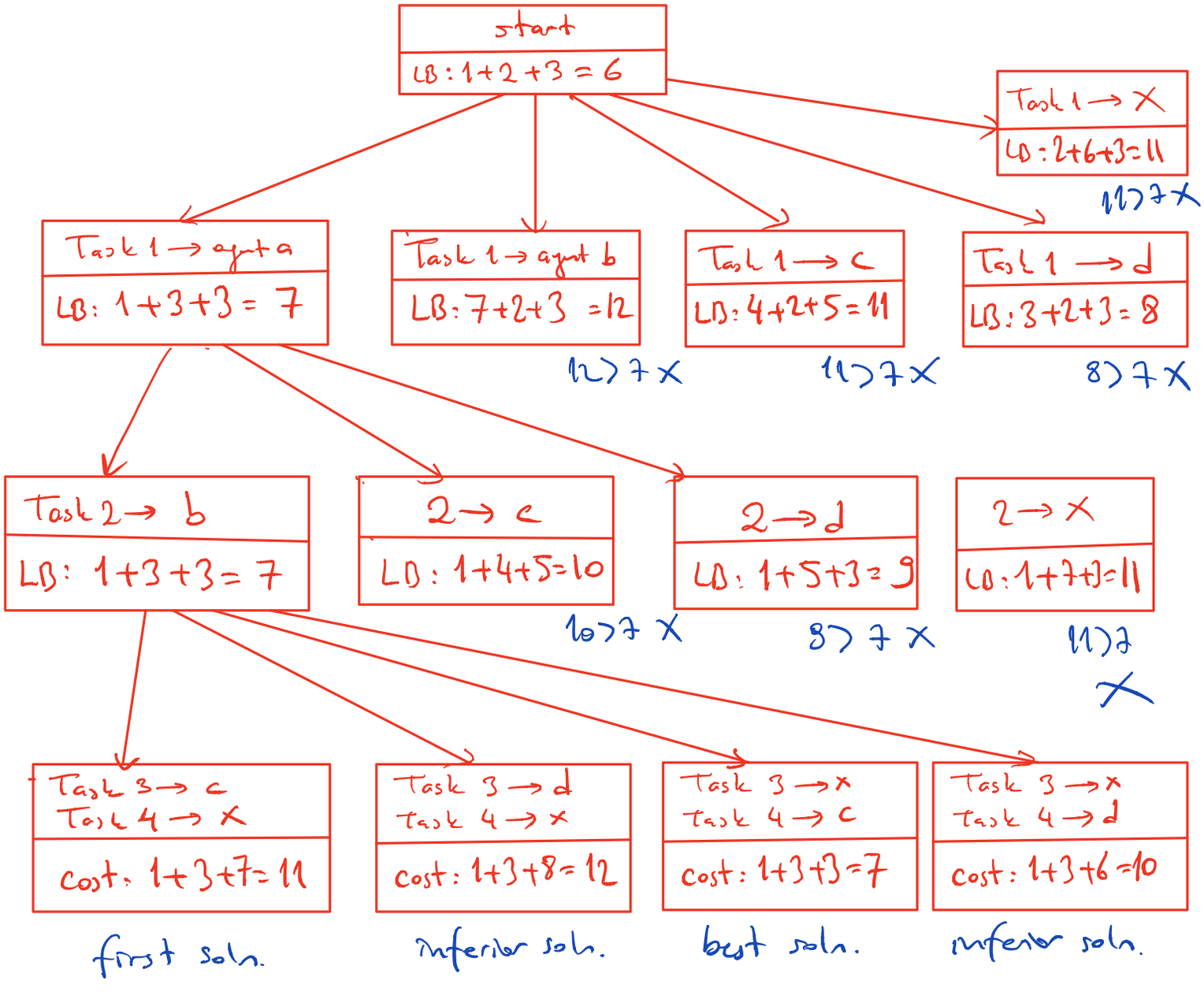
30

Q-5. (~~25~~ pts) Consider a **modified version of Assignment problem**. There are *n agents* and *m* tasks (where $m \leq n+1$). Any agent can be assigned to perform any task, incurring some *cost* that may vary depending on the agent-task assignment. It is required to perform **ALL TASKS EXCEPT ONE** (it is enough to perform m-1 tasks) by assigning exactly one agent to each task in such a way that **the *total cost* of the assignment is minimized**. Note that, some of the agents may not be assigned to any task and stay idle.

(a – 15 pts) For the following utility matrix, define a bounding function and apply a best-first **branch-and-bound algorithm**.

	Task 1	Task 2	Task 3	Task 4
Agent a	1	2	6	5
Agent b	7	3	9	5
Agent c	4	4	7	3
Agent d	3	5	8	6

Bounding function, sum lowest costs excluding one task.



In the question, it is not clear whether single agent can be assigned to multiple tasks or not. So I'll accept that assumption as well.

Name: KEY ID:

(b – ~~10~~ 15 pts) Describe a **greedy algorithm** for the above problem and apply your algorithm for the same instance. What is the **accuracy ratio** for this instance? Show whether the **performance ratio** of your greedy algorithm is unbounded above or not. (Prove your answer).

	Task 1	Task 2	Task 3	Task 4
Agent a	1	2	6	5
Agent b	7	3	9	5
Agent c	4	4	7	3
Agent d	3	5	8	6

Step 1: Among all cost values, choose the one with minimum value, such that the corresponding task is not assigned to any agent. (make that assignment)

Step 2: Repeat Step 1 if # assigned tasks < n-1.

	Task 1	Task 2	Task 3	Task 4
Agent a	(1)	2	6	5
Agent b	7	(3)	9	5
Agent c	4	4	7	(3)
Agent d	3	5	8	6

- Assign agent a to task 1
- Skip agent a - task 2 assignment.
- Assign agent b to task 2
- Assign agent c to task 4.
- Stop.

Accuracy ratio is 1, because we've found the optimal.

In the above example if agent c - task 3, agent c - task 4, agent d - task 3, agent d - task 4 assignment costs were arbitrarily high, the accuracy ratio would be arbitrarily high!

Name: KEY ID:

Q-6. (20 pts) Lower Bound, P, NP, NP-Complete, Approximation.

(a – 7 pts) Suppose that we have n entries of mobile phones in an electronic store. These phones are of 5 different brands. We want to sort these phone entries according to their brands. What is the trivial lower bound of this sorting problem? Is this lower bound tight? Why?

Trivial LB is $\Omega(n)$ because we need to do comparison for all n entries.

$\Omega(n)$ is a tight LB, because we have an algorithm (counting sort) to solve this problem in $O(n)$ time.

(b – 4 pts) Mr. Safi claims that he finds an algorithm with $O(2^n)$ time complexity for TSP problem with a performance ratio of 2 for Euclidian instances. What would be your your comment?

There are already polynomial-time algorithms with performance ratio better than 2.

(c – 9 pts) It is proven that sorting problem can be reduced to Problem X, and Problem X can be reduced to Traveling Salesman Problem (TSP). All reductions can be done in linear (O(n)) time. For each of the following, indicate whether it is **True, False, or Unknown. Describe your reasoning (otherwise your answer will not be graded).**

(i) Problem X may be solved by an algorithm with linear time complexity.

Problem X is at least as hard as sorting problem. So it cannot be solved faster than O(nlogn). FALSE

(ii) Problem X is NP-Complete (or NP-Hard).

UNKNOWN We reduce Problem X to an NP-Complete problem, but we need to reduce an NP-Complete problem to Problem X in order to know this.

(iii) Problem X can be reduced to Knapsack Problem in polynomial time.

TRUE Problem X $\longrightarrow$ TSP $\longrightarrow$ Knapsack.

NP-Complete problems can be reduced to each other. So Problem X can be reduced to Knapsack problem.